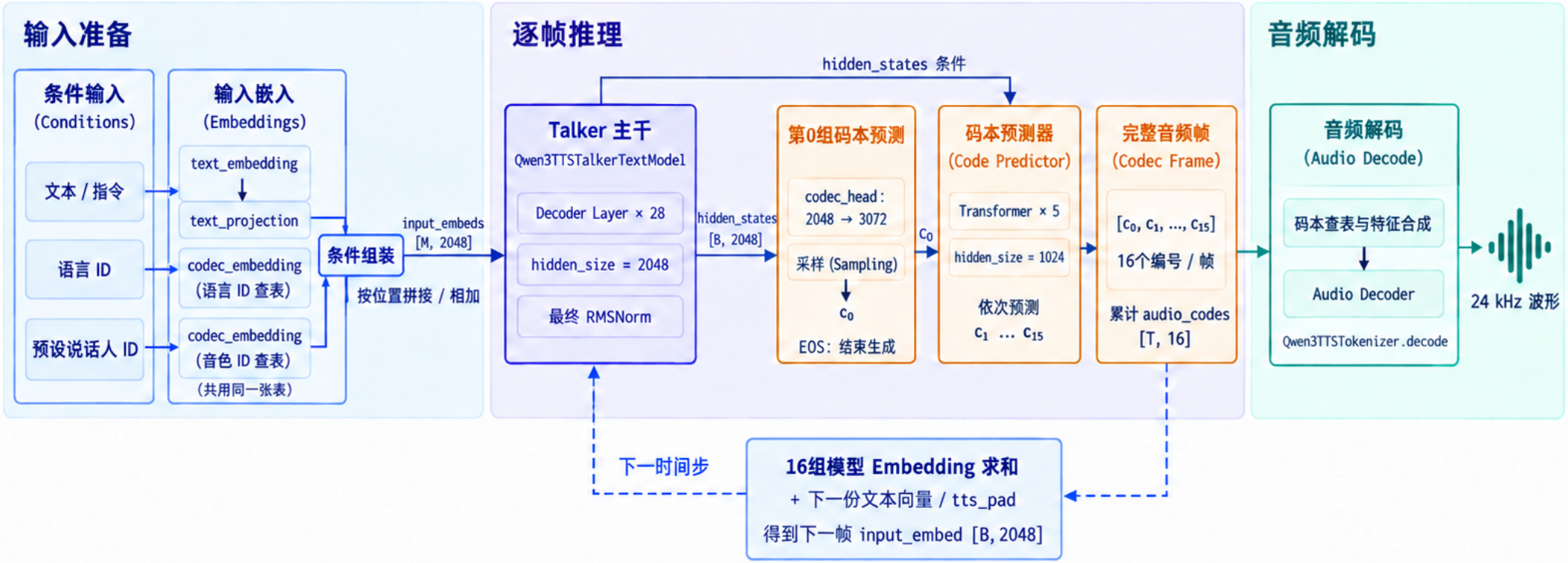
# 01 | Qwen3-TTS CustomVoice：整体生成链路

Qwen/Qwen3-TTS-12Hz-1.7B-CustomVoice

文本与条件 → 音频码本编号 → 音频波形




**A 一帧是什么？**

同一时间步的16个码本编号，
对应 audio_codes 的一行。

**B 两个自回归维度**

Talker：推进音频时间；
Code Predictor：补齐同一帧的码本。

**C 本图范围**

CustomVoice 使用预设音色；
参考语音编码路径见图02。

M：本轮输入 token 数 · B：请求数 · T：音频帧数

# 02 | 输入侧：从参考语音到 Transformer 嵌入

音频特征 → 量化编号 → 模型 Embedding

**参考语音码本路径：Base 的上下文语音克隆(ICL)**　CustomVoice 输入文本；生成音频的反馈可直接从码本编号处接入。

## 一、音频编码与量化（从参考语音到离散码本编号）

**参考语音 (Waveform)**
24 kHz 单声道



**音频编码器 (Audio Encoder)**
卷积下采样 → Transformer → 下采样
按时间压缩得到音频帧

帧级连续特征
[1, 512, T]

**Codec 量化码本 C**
各组 [2048, 256]



匹配时使用

**量化器 (Vector Quantizer)**
投影到量化空间
按码本匹配最近向量
保留有效16组编号

**参考码本编号 (ref_code)**
[ 12, 3, ⋯, 9 ]
[ 8, 21, ⋯, 42 ]
整数 [T, 16]
每行一帧，每列一个码本；
编号仅作示意。

## 二、模型输入嵌入（从码本编号到 Transformer 输入）

生成音频反馈
（可直接接入）

**码本编号**
[ 12, 3, ⋯, 9 ]
[ 8, 21, ⋯, 42 ]
整数 [T, 16]

**模型嵌入查表 (Model Embedding)**
$E_0$ = model.codec_embedding
$E_1 \cdots E_{15}$ = code_predictor.model.codec_embedding
每个编号映射到 2048 维
这是模型输入表 E

**按码本组求和 (Sum)**
$e_t = E_0[c_{t,0}] + \cdots + E_{15}[c_{t,15}]$
音频嵌入 [T, 2048]

**条件组装 (Prompt)**
按位置拼接或相加
文本 / 语言 / 音色条件
input_embeds [M, 2048]

**Talker Transformer**
送入第 1 个 Decoder Layer
Decoder-only Dense Transformer
28 层，hidden_size = 2048

### 小例子：同一个编号，分别查两种表

**量化码本 C [3, 2]**
C = [[0, 0], [1, 0], [0, 1]]
特征 z = [[0.9, 0.1], [0.2, 0.8]]
z 的形状：[2帧, 2维]

**最近向量匹配**
codes = [1, 2]
形状：[2帧]

**模型嵌入表 E [3, 2]**
E = [[0, 0], [3, 1], [1, 4]]
E[codes] = [[3, 1], [1, 4]]
输出形状：[2帧, 2维]

**C 用于量化匹配；**
**E 用于给 Transformer 准备输入。**

简化例子仅用单码本、两帧、二维；主链只展开参考码本分支，音色向量另路准备。

T：帧数；M：提示 token 数。

# 03 | 输出侧：从隐藏状态到音频波形

## hidden_states → Codec Tokens → Codec 特征 → Waveform

hidden_states 条件

**A. Talker 输出**
最终 RMSNorm 后
hidden_states [1, 2048]

**B. 首码本预测**
codec_head：2048 → 3072
logits → 采样(Sampling) → $c_0$
3072含控制符；EOS结束，不送入音频解码。

$c_0$

**C. 码本预测器 (Code Predictor)**
hidden_states 与 $E_0(c_0)$ 作为条件
共享5层 Transformer
依次生成 $c_1 \dots c_{15}$

**D. 一帧完整编号**
codes_t = $[c_0, c_1, \dots, c_{15}]$
形状 [16]

**E. 累计 / 分块**
audio_codes [T, 16]
整数编号

### Qwen3TTSTokenizer.decode：编号 → 连续特征 → 波形

audio_codes [T, 16]
整数编号
(来自上图 E)

第0组 $c_0$
形状 [T]
→ 查 Codec 码本 $C_0$ [2048, 256] → 向量 [T, 256]
→ 输出投影 $P_{sem}$ 256 → 512 → [T, 512]

第1 – 15组
$c_1 \dots c_{15}$
形状 [T, 15]
→ 分别查表，向量求和 $\sum_{g=1}^{15} C_g[c_g]$ → 向量 [T, 256]
→ 输出投影 $P_{ac}$ 256 → 512 → [T, 512]

+ → 量化特征 [1, 512, T] → pre_conv → pre_transformer → 上采样与卷积解码 → 波形 (Waveform)
浮点 [N] · 24 kHz

查表向量256维 → 分支投影512维

每组 Codec 码本：[2048, 256]

沿时间轴展开为连续采样点

### 下一帧反馈（用于生成后续帧）

来自 D 的码本编号
codes_t [16]
→ 模型输入 Embedding 求和
feedback = $\sum E_g[c_g]$
形状 [1, 2048]
→ + 下一份文本向量 / tts_pad
→ 下一轮 Talker

反馈使用模型表 E；
声音还原使用 Codec 表 C。

T：音频帧数；N：波形采样点数。内部增加批维并转置；流式实现按块送入解码器。

# 04 | 核心架构：Transformer 层前后与内部计算

对照《Qwen3-TTS 模型计算示例》①–⑨ · Decoder-only Dense Transformer

## 层前 / 层后

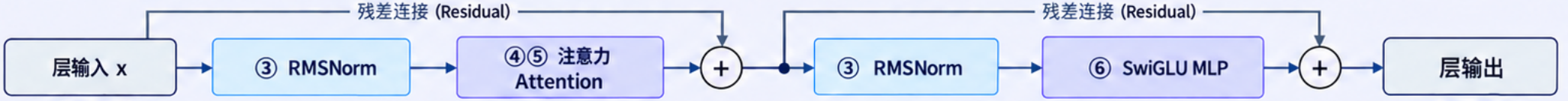



| 组件 | 层数 | 隐藏维度 | MLP中间维度 |
|---|---|---|---|
| Talker | 28 | 2048 | 6144 |
| Code Predictor | 5 | 1024 | 3072 |

⑧ Code Predictor：前缀 [h, t；Eᵤ(cᵢ)] 沿序列维拼接 → 投影2048 → 1024 → 同一套5层网络复用15次，配15个预测头。

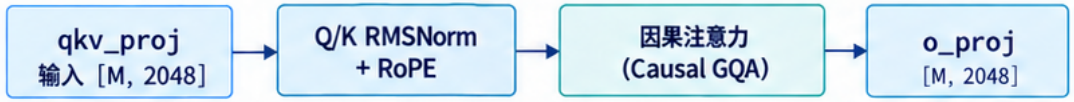
Prefill 取各请求末位置； Decode 每请求取本轮位置

## 单层 Qwen3TTSTalkerDecoderLayer · 数学等价结构




实现中残差相加与 RMSNorm 可融合； MLP后的相加延后到下一层或最终归一化。

## ④⑤ 注意力层(Attention) · Talker 参数




Q：[M, 16, 128]
K / V：[M, 8, 128]

仅处理Q/K，V保持原值

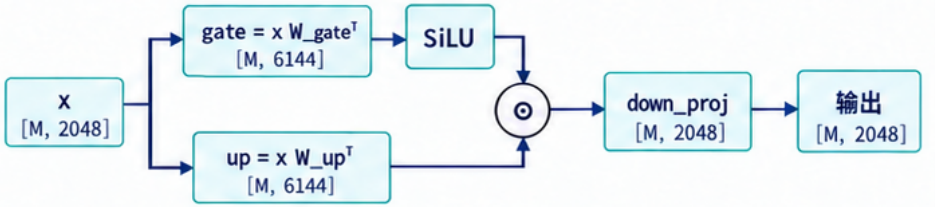
$\text{softmax}(QK^{\top} / \sqrt{128} + \text{mask})\ V$

各头拼接后投影 → [M, 2048]

历史K/V来自 KV cache，位于GPU显存 (GPU memory)。

16个Q头共享8个KV头。

## ⑥ 门控前馈网络(SwiGLU MLP)




gate / up：[M, 6144]； W_gate / W_up：[6144, 2048]； W_down：[2048, 6144]。

## ② 投影小例子：Linear → SiLU → Linear

$x = [[1,2],[2,1]]$

$W_1 = [[1,0],[0,1]]$； $W_2 = [[1,1],[0,1]]$

$y = \text{SiLU}(x\ W_1^{\top})\ W_2^{\top}$

$y \approx [[2.493,1.762],[2.493,0.731]]$

$x$、$W_1$、$W_2$，$y$ 均为 [2,2]

## ③ 残差 + RMSNorm 小例子

$h = [[1,2],[2,1]]$； $r = [[2,2],[2,2]]$

$x = h + r = [[3,4],[4,3]]$； $\gamma = [2,1]$

$y = x / \text{sqrt}(\text{mean}(x^2,\ \text{按行})) \odot \gamma$

$y \approx [[1.697,1.131],[2.263,0.849]]$

h、r、x、y：[2,2]； γ：[2]

数值例子相互独立：2个token、2维、TP=1，省略bias和eps；权重矩阵按[输出维度,输入维度]。 M：输入token数； B：请求数。